

Víncius Santos Anjos da Silva

**Modelagem baseada em agentes paralelizada em CUDA:
O impacto da COVID-19 em áreas urbanas no Brasil**

Universidade Federal Fluminense - UFF

Campus Volta Redonda

Instituto de Ciências Exatas

Orientador: Aquino Lauri de Espíndola

25 de novembro de 2022

Resumo

Para simular uma epidemia em uma população ao longo de um tempo utiliza-se de modelos epidemiológicos, dentre eles está o modelo baseado em agentes (ABM), no qual cada agente é equivalente a um indivíduo na população e possui diversas características, entretanto modelar áreas que possuam grandes populações e diversos parâmetros demanda grande capacidade computacional. Com o intuito de reduzir o tempo das simulações, esse trabalho tem como objetivo implementar uma modelagem baseada em agentes paralelizada utilizando CUDA, uma API que estende as linguagens C/C++ para utilizar os recursos computacionais de GPUs e criar programas paralelizáveis.

Palavras-chaves: Covid-19, modelagem baseada em agentes, programação paralela, GPU, CUDA

Sumário

Sumário	3
1 INTRODUÇÃO	4
2 OBJETIVOS E RESULTADOS ESPERADOS	5
3 FUNDAMENTOS TEÓRICOS	6
3.1 Modelos epidemiológicos compartimentais	6
3.2 Número básico de reprodução	7
3.3 Modelo baseado em agentes (ABM)	7
3.4 Método de Monte Carlo e geração de números aleatórios	9
3.5 Arquitetura de GPUs	9
3.5.1 Organização dos processadores	9
3.5.2 Hierarquia de memória da GPU	10
3.6 Modelo de programação em CUDA	11
3.7 Métricas de desempenho paralelo	12
4 METODOLOGIA	14
4.1 O Modelo	14
4.2 Parâmetros	15
4.3 Implementação em CUDA	16
REFERÊNCIAS	17

1 Introdução

O novo vírus SARS-CoV-2 que causa a Covid-19, se espalha a partir de pequenas partículas líquidas emitidas por tosse, espirros e saliva de pessoas infectadas. A disseminação da doença está principalmente relacionada ao contato próximo de indivíduos em ambientes confinados, sem ventilação adequada e com interação frequente com um número significativo de pessoas. Pelas características do vírus, a evolução da doença se comporta de maneira diferente por região, elementos específicos de cada área como densidade populacional, pirâmide etária, leitos hospitalares e de tratamento intensivo, são fundamentais para como o espalhamento do vírus e sua letalidade ocorrerá em tal região.

Para tentar prever como uma doença se espalhará em uma epidemia são utilizados modelos epidemiológicos, que podem ser determinísticos, através de equações diferenciais, ou estocásticos, que evoluem através de probabilidades. Neste trabalho será feita uma modelagem baseada em agentes, no qual a evolução da epidemia ocorre através da interação de indivíduos que possuem diversas características como idade, sexo e outras que sejam relevantes ao modelo. A presença de diversos parâmetros e o grande número de agentes para algumas áreas, torna necessário um maior poder computacional para a calibração e implementação do modelo, assim sua paralelização pode ser uma forma eficiente de encontrar os resultados do modelo.

GPUs são orientadas a *throughput* em detrimento de latência e logo conseguem realizar tarefas mais simples e em número maior do que CPUs. Com a popularização de GPUs Nvidia no mercado e a existência da API CUDA (Compute Unified Device Architecture), que estende C/C++ para computação em paralelo. A modelagem baseada em agentes em conjunto com a API CUDA torna-se uma boa tentativa de criar um modelo que possa representar diferentes áreas do Brasil e ainda obter uma eficiência, para que cada área não demande muito tempo.

2 Objetivos e Resultados esperados

Este trabalho possui os seguintes objetivos

- Criação de um modelo baseado em agentes, que contemple as características dos indivíduos e das particularidades das áreas modeladas.
- Mostrar como a Covid-19 impacta uma área de acordo com suas características.
- Implementação do modelo em paralelo utilizando CUDA.
- Validação dos resultados obtidos em paralelo com o obtido em serial.
- Benchmark que mostre uma melhora significativa na eficiência do modelo em paralelo implementado

3 Fundamentos teóricos

3.1 Modelos epidemiológicos compartimentais

Para descrever a evolução de uma doença infecciosa em uma população ao longo do tempo, utilizam-se os modelos epidemiológicos compartimentais, nos quais a população é dividida em compartimentos que representam os possíveis estados de cada indivíduo em relação à doença, e a dinâmica da epidemia é descrita pela taxa de transição entre esses compartimentos. O modelo mais elementar é o SIR, no qual a população é dividida em três compartimentos: Suscetíveis (S), aqueles que ainda podem contrair a doença; Infecciosos (I), aqueles que possuem a doença e podem transmiti-la; e Removidos (R), aqueles que se recuperaram ou faleceram e não participam mais da transmissão. Em sua formulação determinística, a evolução de cada compartimento ao longo do tempo é descrita por um sistema de equações diferenciais ordinárias.

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I \quad (3.1)$$

no qual N é o tamanho total da população, β é a taxa de transmissão e γ é a taxa de recuperação, tal que $1/\gamma$ representa o período médio durante o qual o indivíduo permanece infeccioso.

Para doenças nas quais existe um período de latência entre o momento da infecção e o momento em que o indivíduo se torna capaz de transmitir o vírus, o modelo SIR é estendido para o modelo SEIR, no qual é acrescentado o compartimento dos Expostos (E), que representa os indivíduos que já adquiriram a doença, mas ainda passam por um período de incubação e não infectam outros indivíduos.

$$\begin{aligned} \frac{dS}{dt} &= -\beta \frac{SI}{N}, & \frac{dE}{dt} &= \beta \frac{SI}{N} - \sigma E, \\ \frac{dI}{dt} &= \sigma E - \gamma I, & \frac{dR}{dt} &= \gamma I. \end{aligned} \quad (3.2)$$

no qual σ é a taxa de progressão do estado exposto para o estado infeccioso, tal que $1/\sigma$ representa o período médio de latência. A formulação determinística, entretanto, assume uma população homogênea e completamente misturada, na qual todos os indivíduos têm a mesma probabilidade de contato entre si, e logo não captura a estrutura espacial da população, a heterogeneidade entre os indivíduos nem o caráter estocástico do contágio. Para contemplar essas características, utiliza-se a modelagem baseada em agentes, descrita na Seção 3.3, que pode ser entendida como a contraparte estocástica e espacial dos modelos compartimentais.

3.2 Número básico de reprodução

Um dos parâmetros mais importantes na caracterização de uma epidemia é o número básico de reprodução, denotado por R_0 , definido como o número médio de indivíduos que um único caso infecta diretamente em uma população completamente suscetível. O valor de R_0 determina o comportamento inicial da epidemia, uma vez que, quando $R_0 < 1$, cada caso gera em média menos de um novo caso e a doença tende a se extinguir, enquanto, quando $R_0 > 1$, cada caso gera mais de um novo caso e a epidemia se espalha pela população. Nos modelos compartimentais o número básico de reprodução pode ser obtido a partir dos parâmetros da doença; no modelo SIR, por exemplo, $R_0 = \beta/\gamma$, tal que a razão entre a taxa de transmissão e a taxa de recuperação determina o potencial de espalhamento da doença.

Como o número básico de reprodução depende da taxa de transmissão β , que por sua vez está relacionada à infecciosidade da doença e à forma de contato entre os indivíduos, a infecciosidade β do modelo é calibrada de modo que o R_0 resultante corresponda ao valor estimado para a Covid-19, adotado neste trabalho como $R_0 = 3,5$. O procedimento de calibração da infecciosidade é descrito no Capítulo 4.

3.3 Modelo baseado em agentes (ABM)

Modelos epidemiológicos abrangem diversos tipos de modelos compartimentais, que incluem uma variedade de características, que simulam o comportamento de doenças infecciosas ao longo de um tempo em uma população. O modelo compartimental utilizado será o SEIR, no qual a população é dividida em quatro compartimentos: Suscetíveis (S), representados por aqueles que ainda não adquiriram a doença e podem adquiri-la; Expostos (E), representados por aqueles que já adquiriram a doença, mas estão passando por um estado de latência e não infectam outros indivíduos; Infecciosos (I), aqueles que possuem a doença e infectam outros indivíduos; e Removidos (R), aqueles que, ou se recuperaram da doença, ou faleceram.

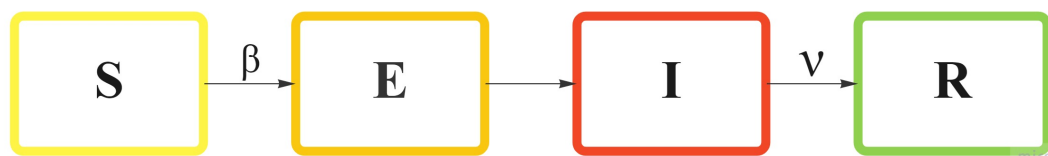


Figura 1 – Fluxograma do funcionamento geral do programa de simulação.

Em ABM, as unidades que interagem entre si, os agentes, apresentam diversas características como estado de saúde, idade, sexo, taxa de isolamento, entre outras que sejam necessárias para a simulação, e são distribuídos de forma heterogênea em uma rede que representa uma população. A evolução da doença na rede depende de como cada agente interage com sua

vizinhança e com outros agentes aleatórios. Em uma rede quadrada, o contato entre os agentes pode ser representado das seguintes formas

- Vizinhança de Von Neumann, os 4 vizinhos (norte, leste, sul, oeste).
- Vizinhança de Moore, todos os 8 vizinhos adjacentes ao agente.
- Contatos aleatórios.

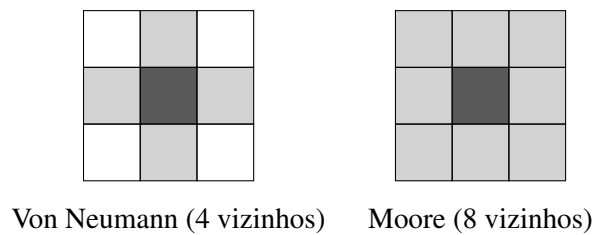


Figura 2 – Vizinhanças em uma rede quadrada. A célula em destaque representa o agente central e as células sombreadas representam os vizinhos considerados no contato: à esquerda, a vizinhança de Von Neumann, com os 4 vizinhos ortogonais; à direita, a vizinhança de Moore, com os 8 vizinhos adjacentes.

Logo, a probabilidade de contágio dependerá da vizinhança e do número de contatos aleatórios, e pode ser representada a partir da seguinte equação.

$$P = 1 - (1 - \beta)^n \quad (3.3)$$

no qual n é a quantidade de infecciosos que tiveram contato com o agente e β é a infecciosidade da doença, que depende de R_0 , parâmetro que representa quantos agentes são infectados a partir de um único caso. Um número aleatório r_n é gerado, e caso $r_n < P$, o agente é infectado.

A dinâmica descrita, na qual cada agente ocupa uma posição fixa em uma rede regular e tem seu estado atualizado a cada passo de tempo em função do estado de sua vizinhança, caracteriza o modelo como um autômato celular probabilístico, no qual as vizinhanças de Von Neumann e de Moore são as mesmas empregadas na teoria de autômatos celulares. Diferentemente de um autômato celular determinístico, no qual a regra de atualização é fixa, no modelo baseado em agentes a transição entre os estados de saúde é governada por probabilidades, e logo cada execução do modelo corresponde a uma realização distinta da epidemia.

3.4 Método de Monte Carlo e geração de números aleatórios

Como a transição dos agentes entre os estados de saúde é decidida de forma probabilística, cada simulação do modelo corresponde a uma única realização do processo estocástico, e logo o resultado de uma simulação isolada não é representativo do comportamento médio da epidemia. Para obter uma estimativa confiável da evolução da doença, utiliza-se o método de Monte Carlo, no qual o modelo é simulado um grande número de vezes de forma independente, e as curvas de prevalência e incidência são obtidas a partir da média sobre todas as simulações realizadas. Quanto maior o número de simulações, menor a flutuação estatística em torno do comportamento médio, uma vez que os desvios de cada realização individual tendem a se compensar.

O método de Monte Carlo depende da geração de números aleatórios, entretanto computadores produzem apenas números pseudoaleatórios, gerados de forma determinística a partir de um valor inicial denominado semente, tal que uma mesma semente produz sempre a mesma sequência de números. Em uma implementação serial uma única sequência de números pseudoaleatórios é suficiente, porém em uma implementação paralela a função geradora precisa ser *thread-safe*, uma vez que diversas threads acessam o gerador simultaneamente, e a utilização de uma mesma sequência por todas as threads introduziria correlações indesejadas entre os agentes. Para contornar essa limitação, CUDA disponibiliza a biblioteca cuRAND, que oferece geradores de números pseudoaleatórios apropriados para a execução em paralelo.

3.5 Arquitetura de GPUs

3.5.1 Organização dos processadores

Enquanto CPUs são orientadas para realizar operações mais complexas, mas com baixa latência, GPUs são orientadas para maior *throughput*, realizam operações mais simples, mas processam bastantes tarefas ao mesmo tempo. Essas características tornam CPUs excelentes para tarefas em serial, enquanto GPUs são ótimas para tarefas paralelizáveis. Essa diferença ocorre devido a suas arquiteturas, CPUs têm poucos *cores* que precisam de memória cache para diminuir a latência, enquanto GPUs não se preocupam tanto com latência, então possuem menos transístores para memória cache em benefício de mais transístores que realizarão processamento de tarefas, dependendo da família da GPU, centenas e até milhares de *cores* estão presentes.

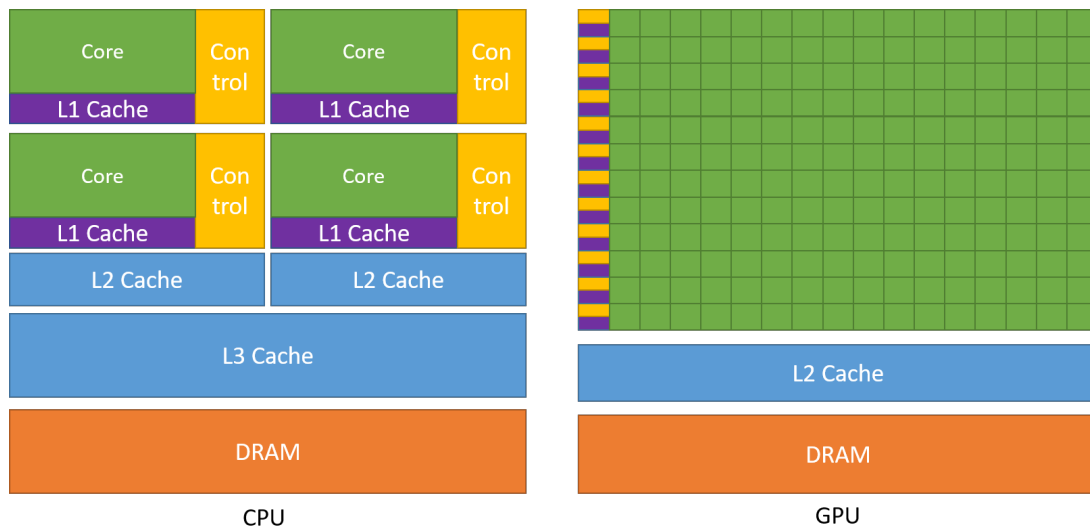


Figura 3 – Arquitetura de uma CPU e de uma GPU.

GPUs contêm SMs (*streaming multiprocessors*), que contêm CUDA *cores* que realizarão as tarefas, cache L1, memória compartilhada entre os processadores dentro do SM e cache L2. Para obter maior *throughput*, os *cores* nos SMs são SIMT (*single instruction multiple thread*), para cada tarefa necessária, um grupo de 32 *cores* realizará a mesma tarefa, esses grupos são chamados de *warps*, e o processamento das tarefas será sempre realizado pela execução dos *warps*.

3.5.2 Hierarquia de memória da GPU

Além da grande quantidade de *cores*, o desempenho de um programa executado em GPU depende fortemente da forma como os dados são acessados na memória, organizada em uma hierarquia de níveis com diferentes capacidades e latências. No nível mais próximo dos *cores* estão os registradores, privados de cada thread e de acesso mais rápido; em seguida está a memória compartilhada, acessível por todas as threads de um mesmo bloco e localizada dentro do SM, de baixa latência; e, no nível mais distante, está a memória global, de maior capacidade, porém de maior latência, na qual residem os dados copiados do *host* para o *device*. Entre a memória global e os SMs encontram-se ainda as memórias cache L1, privada de cada SM, e L2, compartilhada entre todos os SMs, que armazenam os dados acessados recentemente com o intuito de reduzir o número de acessos à memória global.

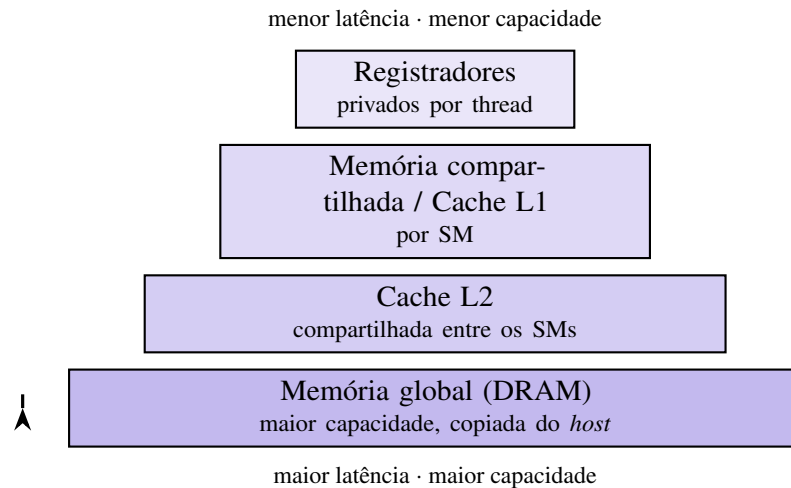


Figura 4 – Hierarquia de memória de uma GPU. Do topo para a base, a capacidade e a latência de acesso aumentam, enquanto a proximidade em relação aos *cores* diminui.

O acesso à memória global é mais eficiente quando é coalescido, ou seja, quando threads de uma mesma warp acessam posições contíguas de memória, que podem então ser atendidas em uma única transação. Em diversas aplicações o desempenho não é limitado pela capacidade de processamento, mas pela largura de banda da memória, situação na qual o programa é dito limitado por memória (*memory-bound*), e logo a forma como os dados são estruturados e acessados torna-se determinante para o desempenho. Em particular, estruturas de dados compactas, que reduzem o volume de dados transferidos e permitem que o conjunto de dados em uso seja mantido na memória cache L2, podem proporcionar ganhos expressivos de desempenho.

3.6 Modelo de programação em CUDA

CUDA é uma API (*Application Programming Interface*) que permite a criação de programas em C/C++ em GPUs Nvidia que sejam compatíveis. Em CUDA, o programador cria funções semelhantes às funções de C/C++; essas funções, chamadas de kernels, realizam N tarefas para as N CUDA threads disponíveis na GPU utilizada. Um conjunto de threads é chamado de "blocks" e um conjunto de "blocks" forma um "grid", esses conjuntos podem ter até 3 dimensões, e podem ser identificados com variáveis nativas dadas por: **threadIdx**, **blockIdx**, **blockDim** e **gridDim**. Manipulando essas variáveis, o programador consegue acessar e realizar tarefas para todas as threads.

Ao desenvolver programas utilizando a arquitetura CUDA, está-se envolvido em uma computação heterogênea, uma vez que o programa faz uso da interação entre a unidade central de processamento (CPU), também conhecida como *host*, e a unidade de processamento gráfico (GPU), denominada *device*. No *host*, o código é responsável por realizar as tarefas que precisam

ser feitas em serial, alocações de memória, copiar os dados da memória da CPU para a GPU e então chamar os kernels para realizar as tarefas no device. Na GPU, cada SM realizará as tarefas por bloco e, após a finalização das execuções, o *host* copia os resultados obtidos da memória da GPU para a CPU. A Figura 5 representa como funciona um programa feito em CUDA.

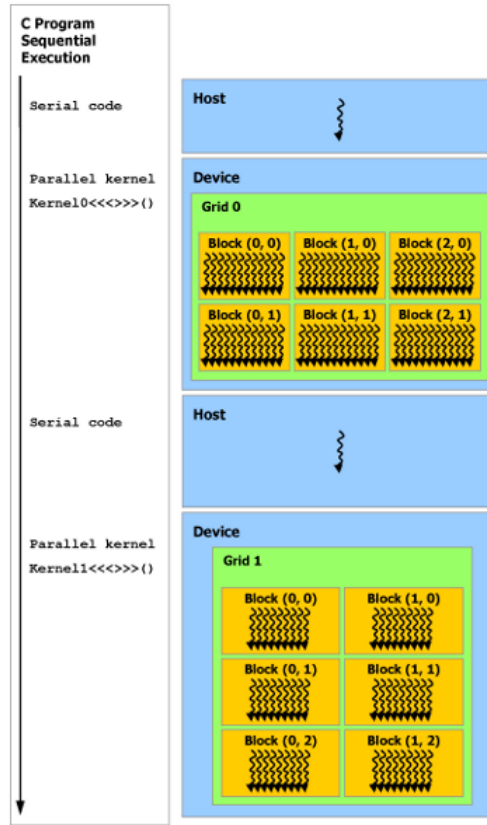


Figura 5 – Execução de um programa feito em CUDA.

3.7 Métricas de desempenho paralelo

Para avaliar o ganho obtido com a paralelização de um programa, utilizam-se métricas que comparam o desempenho da implementação paralela com o da implementação serial. A principal delas é a aceleração (*speedup*), definida como a razão entre o tempo de execução da implementação serial e o tempo de execução da implementação paralela.

$$S = \frac{T_{serial}}{T_{paralelo}} \quad (3.4)$$

no qual T_{serial} é o tempo de execução do programa em um único processador e $T_{paralelo}$ é o tempo de execução do programa paralelizado. Uma segunda métrica é a eficiência, definida como a razão entre a aceleração obtida e o número de unidades de processamento utilizadas.

$$E = \frac{S}{p} \quad (3.5)$$

no qual p é o número de unidades de processamento, tal que a eficiência indica o quanto cada unidade contribui, em média, para a aceleração total.

A aceleração que pode ser obtida, entretanto, é limitada pela fração do programa que precisa ser executada de forma serial, relação expressa pela Lei de Amdahl.

$$S = \frac{1}{f + \frac{1-f}{p}} \quad (3.6)$$

no qual f é a fração do programa que permanece serial, tal que, à medida que o número de unidades de processamento p aumenta, a aceleração tende ao limite $1/f$, e logo a parcela serial do programa determina o ganho máximo que pode ser alcançado, independentemente da quantidade de *cores* disponíveis. Em GPUs, nas quais milhares de threads são executadas simultaneamente, a aceleração é frequentemente medida de forma empírica, comparando-se diretamente o tempo de execução da implementação paralela com o da implementação serial de referência.

4 Metodologia

4.1 O Modelo

A modelagem baseada em agentes utilizada será uma modificação do modelo compartimental SEIR, os indivíduos podem ter os seguintes estados de saúde, suscetíveis (X), expostos (E), pré-sintomáticos (P), assintomáticos (A), sintomático leve (S_l), sintomático moderado (S_m), sintomático severo (S_s), hospitalizado (H), internados na UTI (UTI), recuperados (R) e mortos (M). A Figura 6 abaixo demonstra como ocorre os possíveis movimentos dentre os estados de saúde.

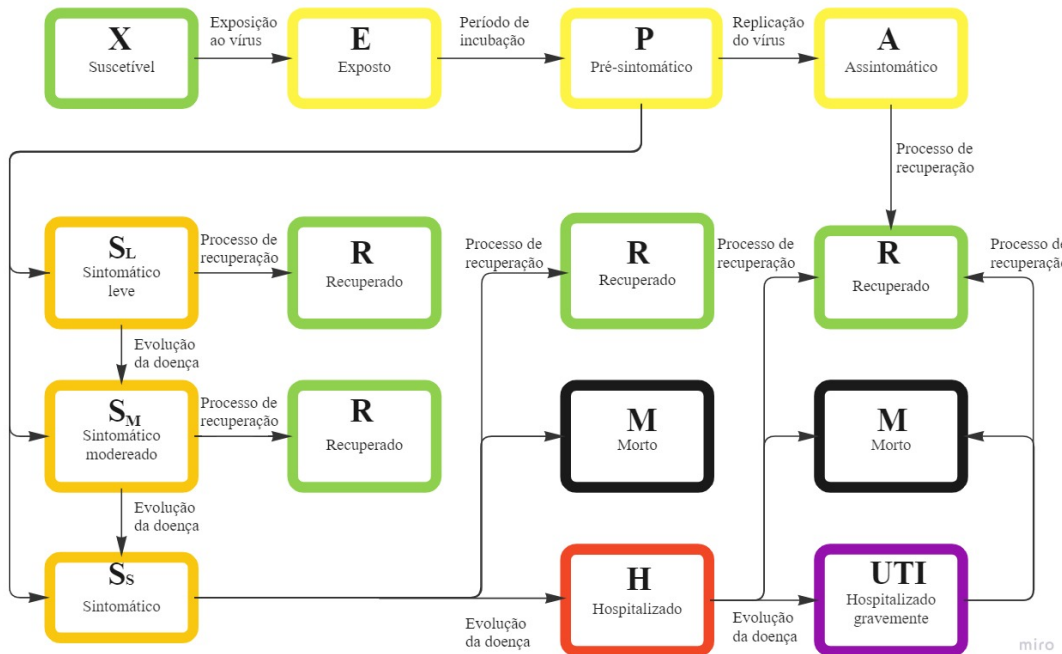


Figura 6 – Fluxograma do movimento entre estados de saúde.

O indivíduo suscetível (X) entra em contato com o vírus e torna-se exposto (E), após o período de incubação torna-se pré-sintomático (P), a replicação do vírus ocorre e o indivíduo ou torna-se assintomático (A) e se recupera da doença, ou a doença evolui e o indivíduo se torna infeccioso, o indivíduo então pode ter sintomas leves (S_l), moderados (S_m) ou severos (S_s), caso não se recupere, será hospitalizado (H) e em casos mais graves internados em unidades de tratamento intensivo (UTI), na UTI o indivíduo ou se recupera, ou morre devido a doença.

No modelo cada indivíduo ocupa uma posição em uma matriz quadrada de lado L com população total $N = L \times L$, tal que N é referente a cada cidade analisada. No início de cada simulação a população é iniciada com todos indivíduos com estado suscetível (X),

exceto 5 indivíduos que estarão em estado pré-sintomático, os indivíduos infectados entrarão em contato com sua vizinhança, Von Neumann para as cidades de baixa densidade populacional ou Moore para cidades de alta densidade, além dos contatos com outros indivíduos aleatórios. A probabilidade de infecção dos indivíduos suscetíveis é dada pela equação 3.3, um número aleatório r é gerado e caso $r < P$, o indivíduo é infectado.

4.2 Parâmetros

Para cada cidade a ser utilizada no modelo, a calibração da infecciosidade β é necessária para que seja padrão em todas as cidades que $R_0 = 3.5$, a calibração é feita utilizando a média de indivíduos infectados a partir de um único caso para diferentes valores de β até que se encontre o valor que mais se aproxima ao desejado. É necessário também mudar os parâmetros que são específicos de cada cidade, são eles: tamanho da população, densidade populacional, contatos aleatórios, leitos de hospital e leitos de UTI. A Tabela 1 mostra as características de 4 cidades, São Paulo (SP), Manaus (AM), Brasília (DF) e a favela da Rocinha (RJ).

Área	Densidade populacional	L	Leitos/Pop	Leitos de UTI/Pop	Contatos aleatórios
São Paulo	Alta	3355	0,00247452	0,00043782	2-19
Rocinha	Alta	264	0,00055111	0,00014592	2-120
Brasília	Baixa	1604	0,00260879	0,00040114	2
Manaus	Baixa	1343	0,00187124	0,00027858	2

Tabela 1 – Parâmetros de 4 diferentes áreas do Brasil.

Já os parâmetros relacionados à doença são comuns a todas as áreas modeladas. A Tabela 2 mostra a duração, em dias, o período de cada estado de saúde, a Tabela 3 mostra a probabilidade de transição entre os estados de saúde.

Parâmetro	Descrição	Duração (dias)
τ_l	Período de latência	0 - 27
τ_P	Pré-sintomático	0 - 14
τ_A	Assintomático	0 - 7
τ_{S_l}	Sintomático leve	0 - 14
τ_{S_m}	Sintomático moderado	0 - 28
τ_{S_s}	Sintomático severo	0 - 4
τ_H	Hospitalização	7 - 45
τ_{UTI}	UTI	10 - 60

Tabela 2 – Parâmetros de duração de cada estado de saúde.

Parâmetro	Transição entre estados	Probabilidade
$P_{(P \rightarrow A)}$	Pré-sintomático para assintomático	0.5
$P_{(P \rightarrow S_l)}$	Pré-sintomático para sintomático leve	0.6
$P_{(P \rightarrow S_m)}$	Pré-sintomático para sintomático moderado	0.2
$P_{(P \rightarrow S_s)}$	Pré-sintomático para sintomático severo	0.2
$P_{(S_l \rightarrow S_m)}$	Sintomático leve para sintomático moderado	0.1
$P_{(P \rightarrow A)}$	Sintomático severo para hospitalização	0.75
$P_{(P \rightarrow A)}$	Internação em UTI	0.25

Tabela 3 – Probabilidades de transição entre estados de saúde.

4.3 Implementação em CUDA

Para implemetação em CUDA mudanças nas redes serão necesárias para que o seu tamanho seja múltiplo de 32, e assim não causar divergência entre os warps. Como o modelo gera números aleatórios, em um programa em paralelo a função rand utilizada em C não é *thread safe*, contudo CUDA oferece, através da biblioteca cuRAND, alguns pseudo geradores de números aleatórios. Com números aleatórios que sejam paralelizaveis e uma paralelização da rede de modo que as vizinhanças não sejam desfeitas, uma paralelização do modelo baseado em agentes deve ser bem sucedida.

Referências

- [1] Merrill R. Introduction to Epidemiology. Jones & Bartlett Learning: London, United Kingdom, 2010.
- [2] Keeling, M.J. and Rohani, P. (2008) Modeling infectious diseases in humans and animals. Princeton: Princeton University Press.
- [3] Sanders, J. and Kandrot, E. (2011) Cuda by example: An introduction to general-purpose GPU programming. Upper Saddle River: Addison-Wesley.
- [4] Cheng, J., Grossman, M. and McKercher, T. (2014) Professional CUDA® C programming. Indianapolis, IN: Wrox, a Wiley brand.
- [5] http://cnes2.datasus.gov.br/Mod_Ind_Tipo_Leito.asp?VEstado=00
- [6] <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [7] <https://docs.nvidia.com/cuda/curand/index.html>